Name: Nelson Puthuraja Nadar **Roll no.:** 21 **Class:** SE/INFT-B **Batch:** B-1

a. Answer the following:

● Give differences between Git and GitHub

Aspect	Git	GitHub
Type	Distributed version control system	Web based hosting platform for Git repositories
Usage	Used to track changes in source code locally	Used to host and manage Git repositories online
Internet Requirement	Does not require internet for most operations	Requires internet to access remote repositories
Installation	Installed on local system	Accessed through a web browser or application
Purpose	Manages version history and code changes	Enables collaboration, code sharing, and project management
Collaboration	Limited to local or manually connected repositories	Provides pull requests, issues, and team collaboration
Storage	Stores repositories on local machine	Stores repositories on cloud servers
Access Control	Managed locally by user	Provides user authentication and permission control
Examples	Git CLI	GitHub, GitHub Desktop

● What is a Git Cheat sheet?

A Git Cheat Sheet is a concise reference guide that lists commonly used Git commands along with their functions. It helps users quickly recall syntax and usage of commands such as creating repositories, staging files, committing changes, branching, merging, and pushing code to remote repositories. Git Cheat Sheets are especially useful for beginners and during practical implementation.

- Attach sample Git Cheat sheet

GitHub

Git Cheat Sheet

Git is the open source distributed version control system that facilitates GitHub activities on your laptop or desktop. This cheat sheet summarizes commonly used Git command line instructions for quick reference.

Install

GitHub for Windows
https://windows.github.com

GitHub for Mac
https://mac.github.com

Git for All Platforms
http://git-scm.com

Git distributions for Linux and POSIX systems are available on the official Git SCM web site.

Configure tooling

Configure user information for all local repositories

`$ git config --global user.name "[name]"`
Sets the name you want attached to your commit transactions

`$ git config --global user.email "[email address]"`
Sets the email you want attached to your commit transactions

`$ git config --global color.ui auto`
Enables helpful colorization of command line output

Branches

Branches are an important part of working with Git. Any commits you make will be made on the branch you're currently "checked out" to. Use `git status` to see which branch that is.

`$ git branch [branch-name]`
Creates a new branch

`$ git checkout [branch-name]`
Switches to the specified branch and updates the working directory

`$ git merge [branch]`
Combines the specified branch's history into the current branch. This is usually done in pull requests, but is an important Git operation.

`$ git branch -d [branch-name]`
Deletes the specified branch

Create repositories

When starting out with a new repository, you only need to do it once; either locally, then push to GitHub, or by cloning an existing repository.

`$ git init`
Turn an existing directory into a git repository

`$ git clone [url]`
Clone (download) a repository that already exists on GitHub, including all of the files, branches, and commits

The .gitgnore file

Sometimes it may be a good idea to exclude files from being tracked with Git. This is typically done in a special file named `.gitignore`. You can find helpful templates for `.gitignore` files at github.com/github/gitignore.

Synchronize changes

Synchronize your local repository with the remote repository on GitHub.com

`$ git fetch`
Downloads all history from the remote tracking branches

`$ git merge`
Combines remote tracking branch into current local branch

`$ git push`
Uploads all local branch commits to GitHub

`$ git pull`
Updates your current local working branch with all new commits from the corresponding remote branch on GitHub. `git pull` is a combination of `git fetch` and `git merge`

GitHub Git Cheat Sheet

Make changes

Browse and inspect the evolution of project files

`$ git log`
Lists version history for the current branch

`$ git log --follow [file]`
Lists version history for a file, including renames

`$ git diff [first-branch]...[second-branch]`
Shows content differences between two branches

`$ git show [commit]`
Outputs metadata and content changes of the specified commit

`$ git add [file]`
Snapshots the file in preparation for versioning

`$ git commit -m "[descriptive message]"`
Records file snapshots permanently in version history

Redo commits

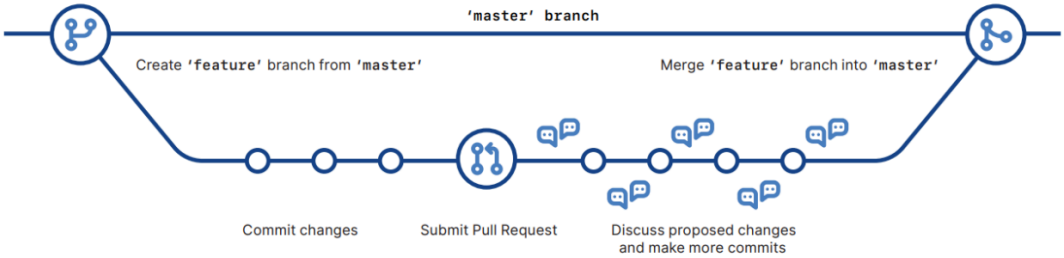
Erase mistakes and craft replacement history

`$ git reset [commit]`
Undoes all commits after [commit], preserving changes locally

`$ git reset --hard [commit]`
Discards all history and changes back to the specified commit

CAUTION! Changing history can have nasty side effects. If you need to change commits that exist on GitHub (the remote), proceed with caution. If you need help, reach out at github.community or contact support.

GitHub Flow



Glossary

git: an open source, distributed version-control system
GitHub: a platform for hosting and collaborating on Git repositories
commit: a Git object, a snapshot of your entire repository compressed into a SHA
branch: a lightweight movable pointer to a commit
clone: a local version of a repository, including all commits and branches
remote: a common repository on GitHub that all team member use to exchange their changes
fork: a copy of a repository on GitHub owned by a different user
pull request: a place to compare and discuss the differences introduced on a branch with reviews, comments, integrated tests, and more
HEAD: representing your current working directory, the HEAD pointer can be moved to different branches, tags, or commits when using `git checkout`

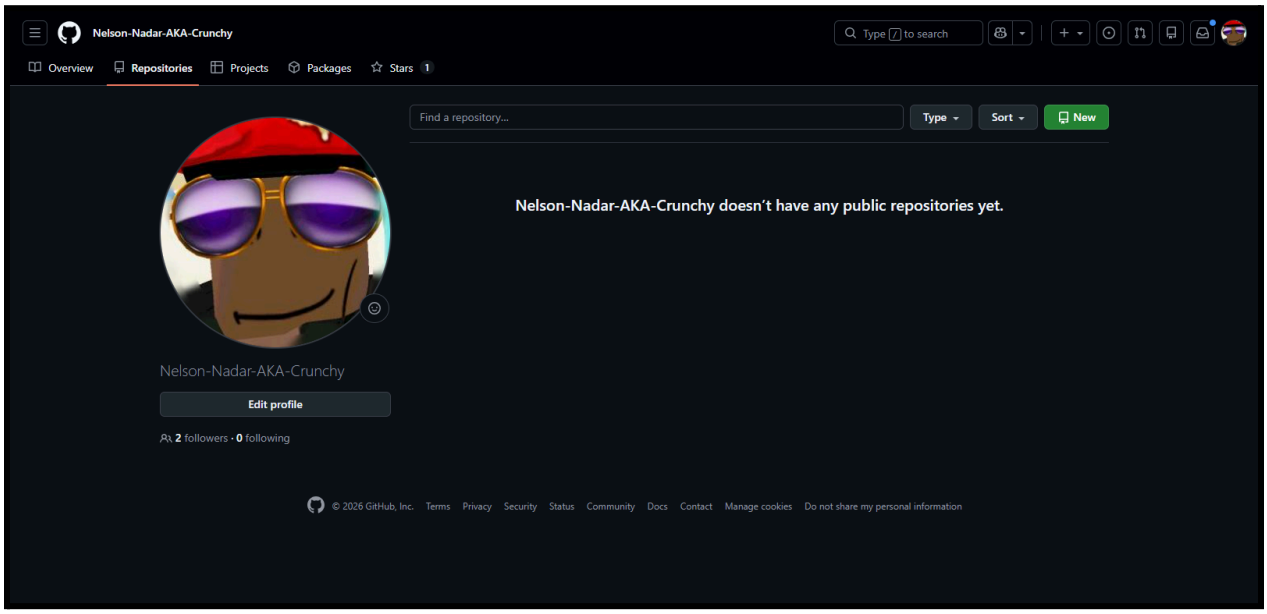
GitHub Training

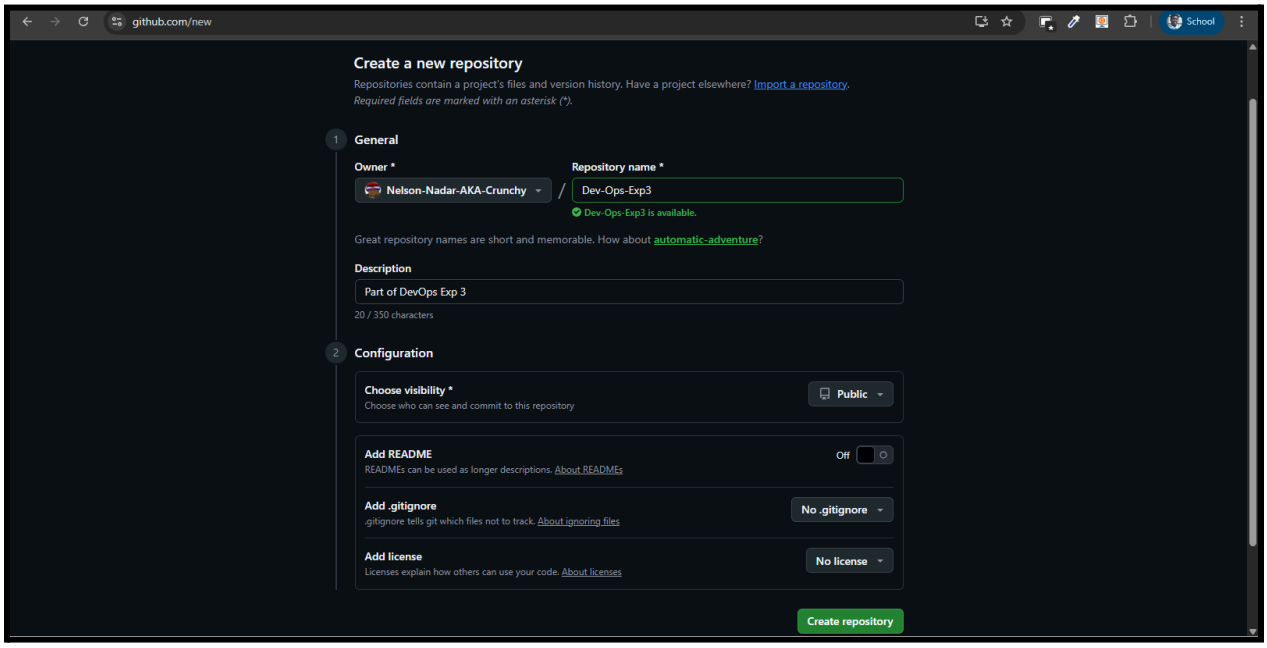
Want to learn more about using GitHub and Git?
Email the Training Team or visit our web site for learning event schedules and private class availability.

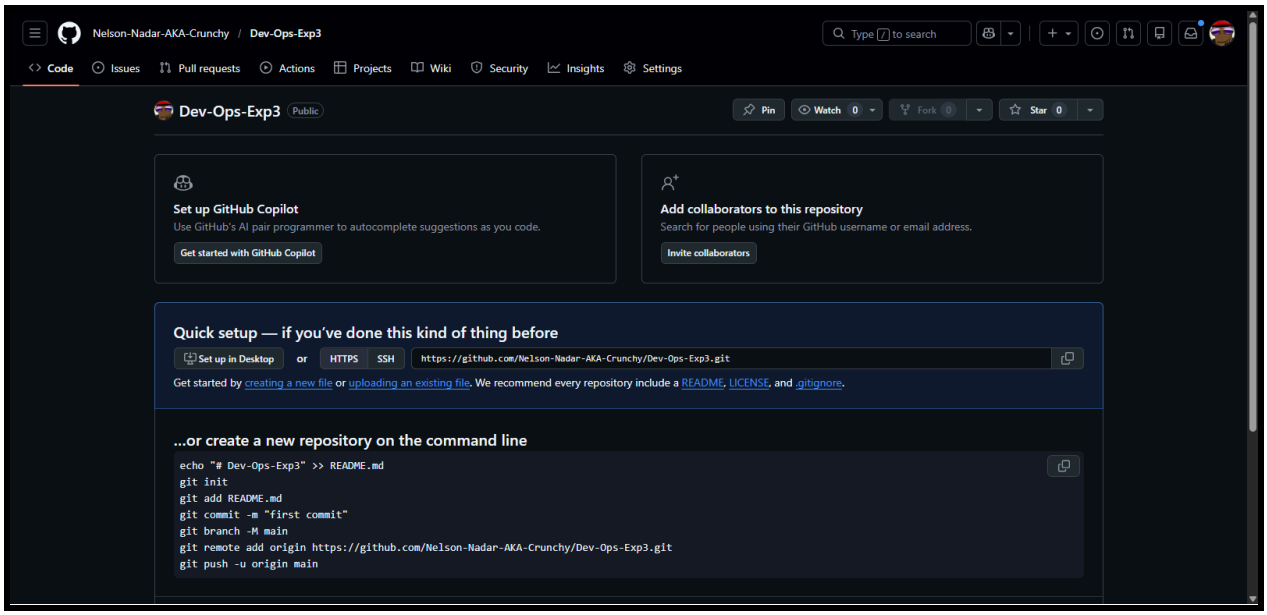
services@github.com
services.github.com

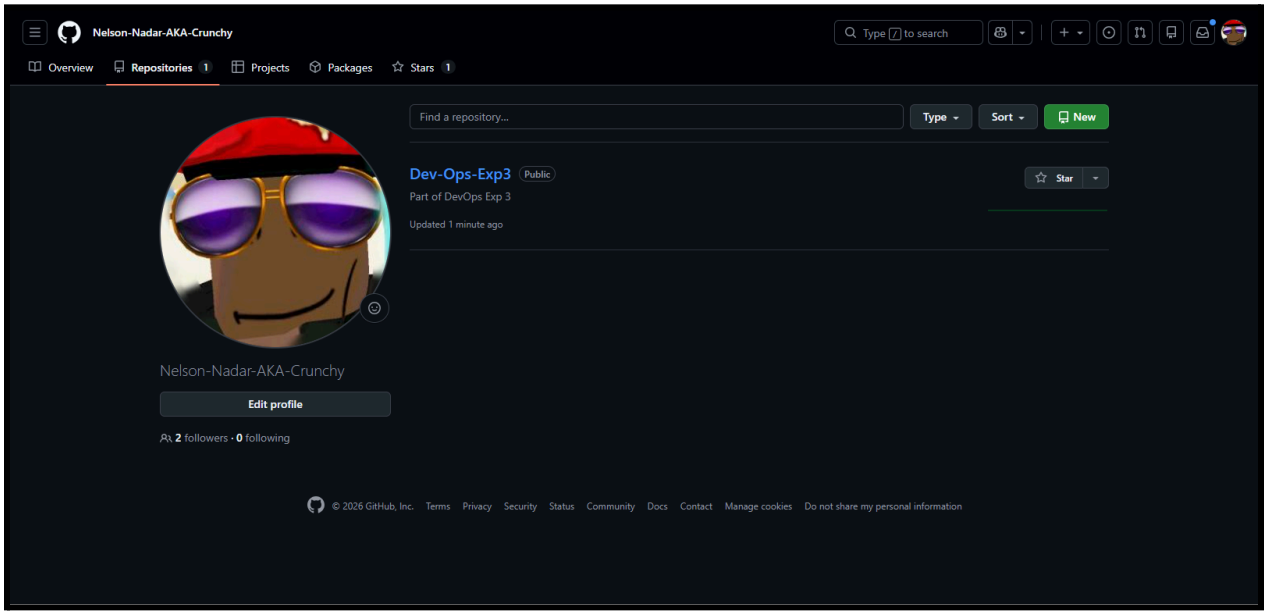
b. Execute following on Git and GitHub (Refer the shared material) and attach screenshots:

1. GitHub repository creation









2. Execution of following Git commands along with screenshots of folder contents corresponding GitHub repository contents

1. git remote add origin

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git remote add origin https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git
```

2. git remote show origin

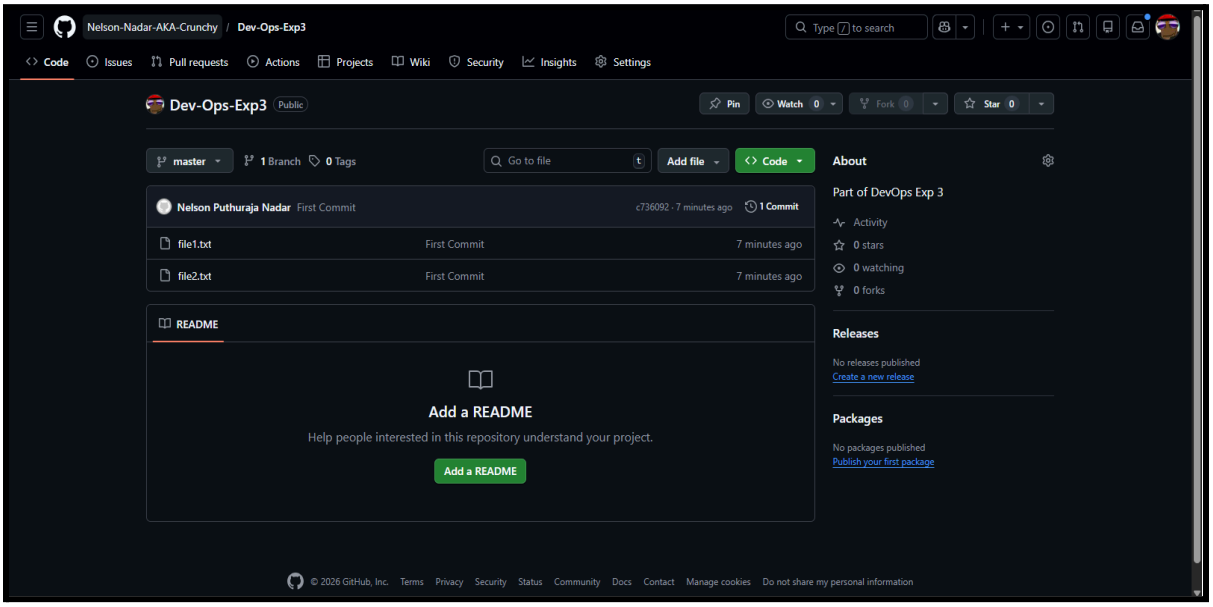
```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git
  Push  URL: https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git
  HEAD branch: (unknown)
```

3. git push origin master

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git commit -m "First Commit"
[master (root-commit) c736092] First Commit
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1.txt
 create mode 100644 file2.txt
```

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git log
commit c7360926c57cdc50a64cdf4bb6b767af526b09a8 (HEAD -> master)
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Feb 3 12:49:33 2026 +0530

    First Commit

MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$
```

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git push origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 239 bytes | 239.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git
 * [new branch]      master -> master
```



4. git fetch origin master

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git fetch origin master
From https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3
 * branch            master     -> FETCH_HEAD
```

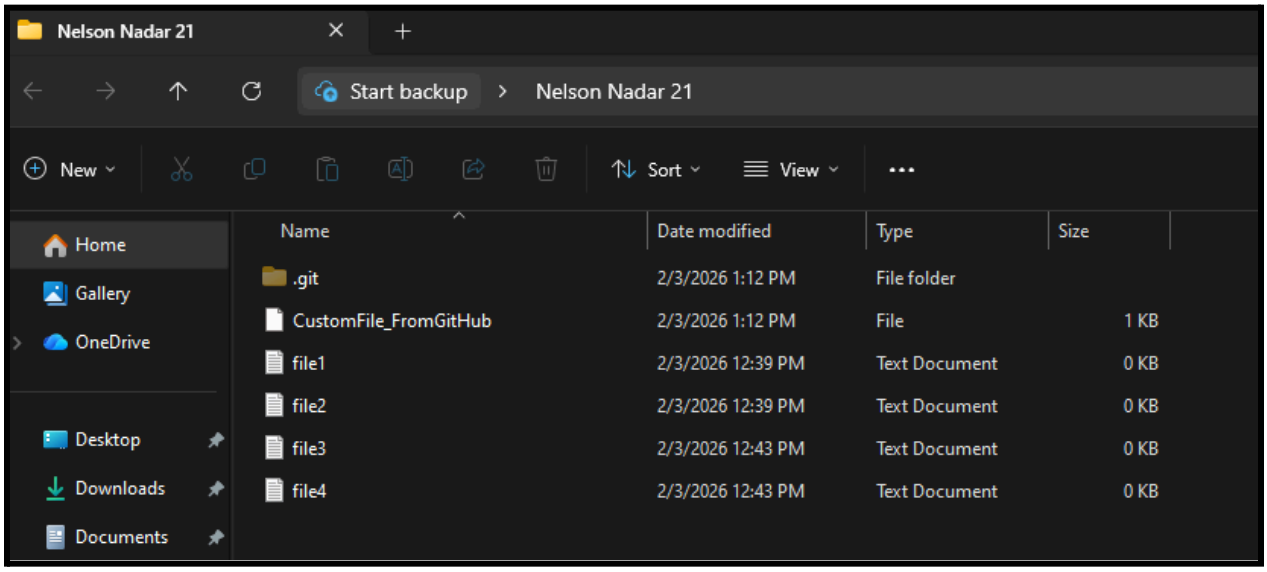
5. git merge origin/master

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git merge origin/master
Already up to date.
```

6. git pull origin master

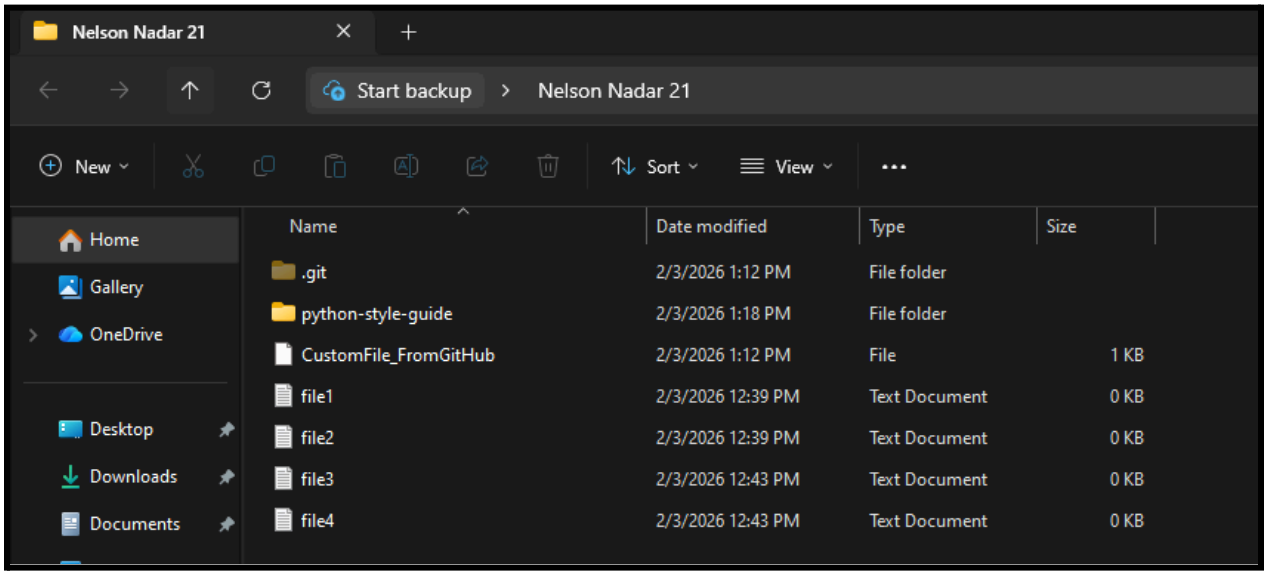
```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git pull origin master
From https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3
 * branch            master     -> FETCH_HEAD
Already up to date.
```

```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git pull origin master
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 977 bytes | 61.00 KiB/s, done.
From https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3
 * branch            master     -> FETCH_HEAD
   a03ed54..6ccc93a  master     -> origin/master
Updating a03ed54..6ccc93a
Fast-forward
 CustomFile_FromGitHub | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 CustomFile_FromGitHub
```



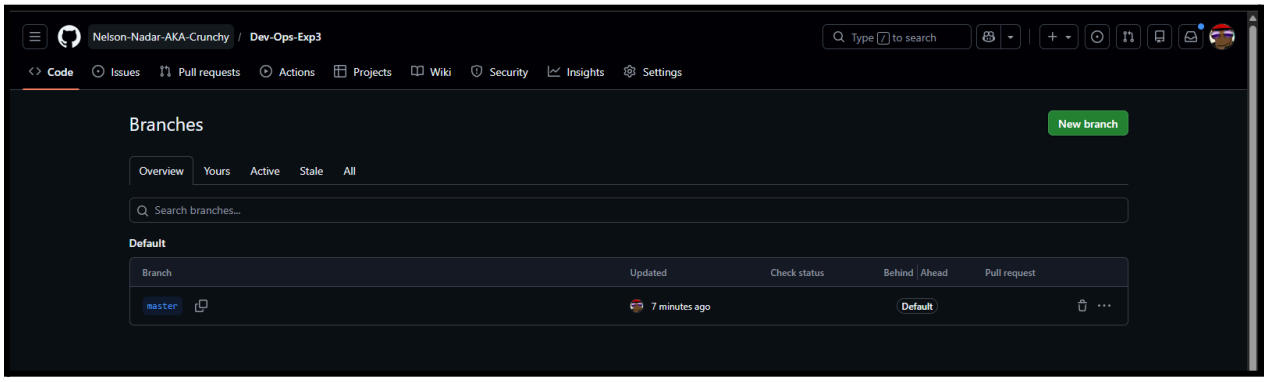
7. git clone

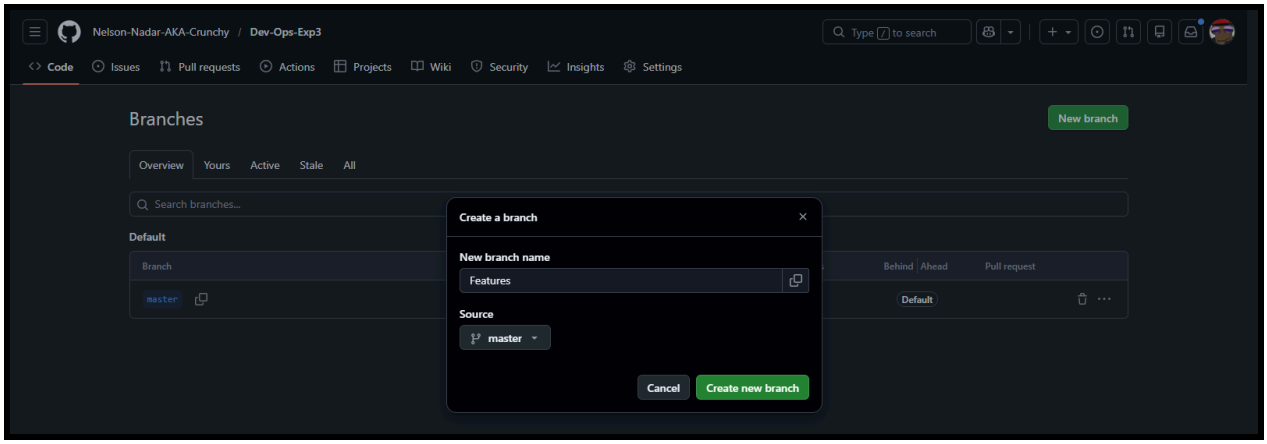
```
MOAD@IT306A-02 MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git clone https://github.com/kenshoo/python-style-guide.git
Cloning into 'python-style-guide'...
remote: Enumerating objects: 138, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 138 (delta 1), reused 0 (delta 0), pack-reused 136 (from 2)
Receiving objects: 100% (138/138), 68.28 KiB | 6.83 MiB/s, done.
Resolving deltas: 100% (39/39), done.
```

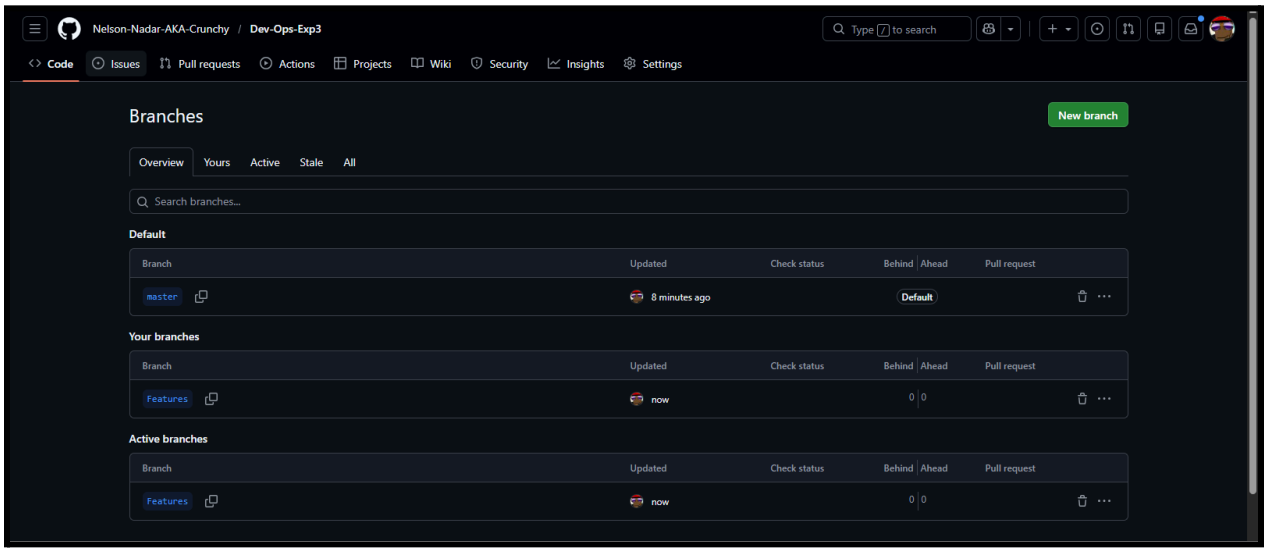


3. On GitHub repository:

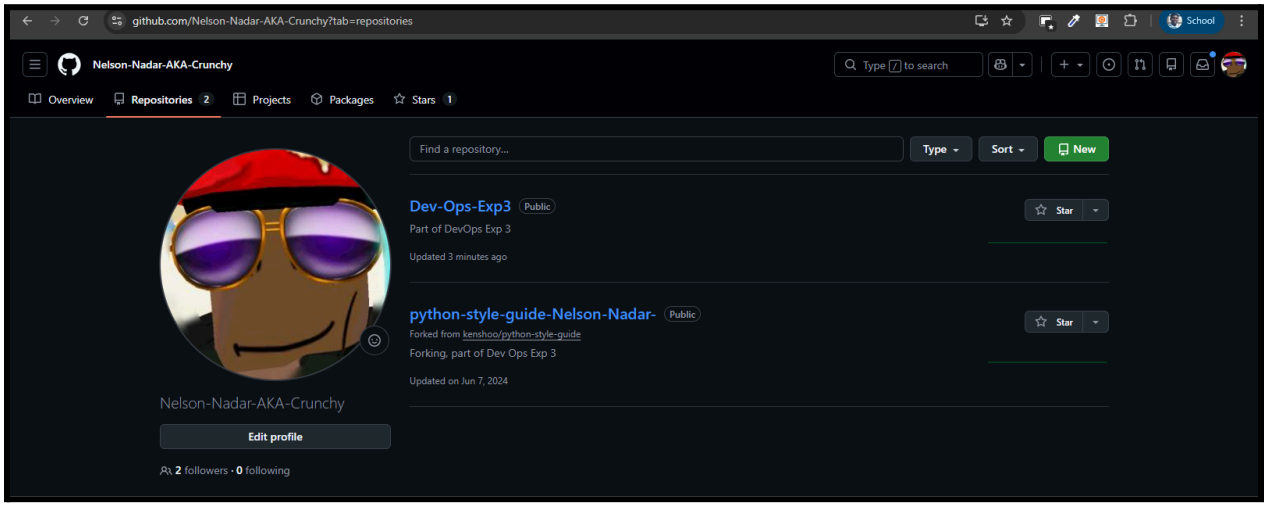
1. New branch creation in own repository

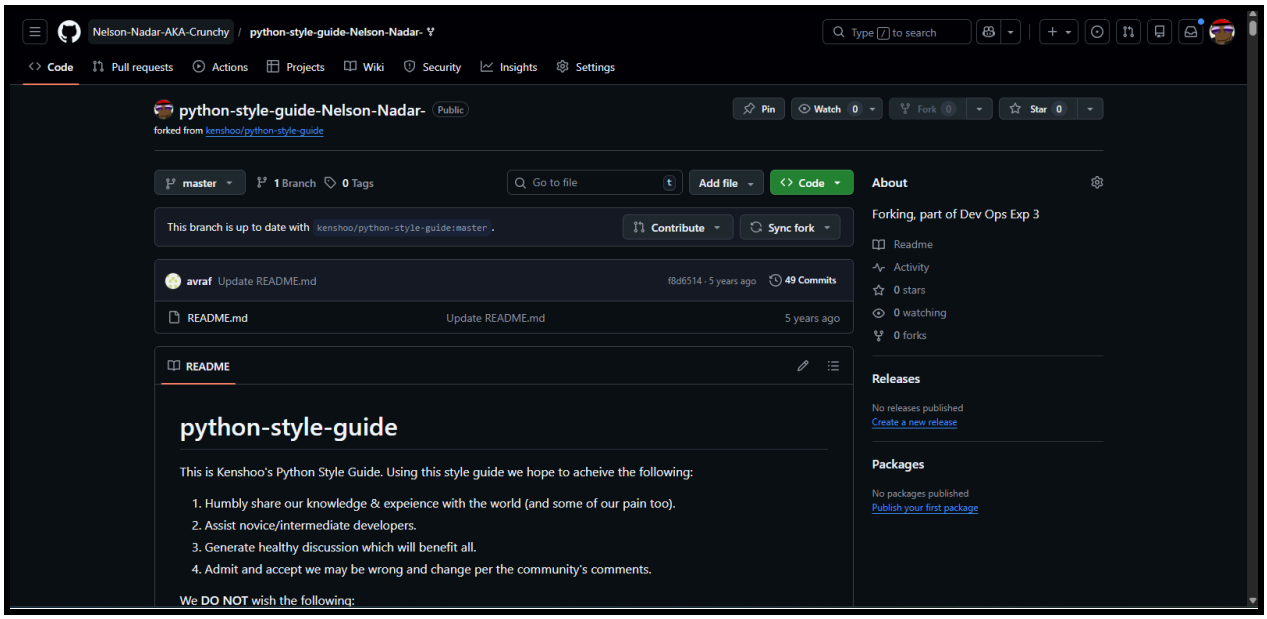




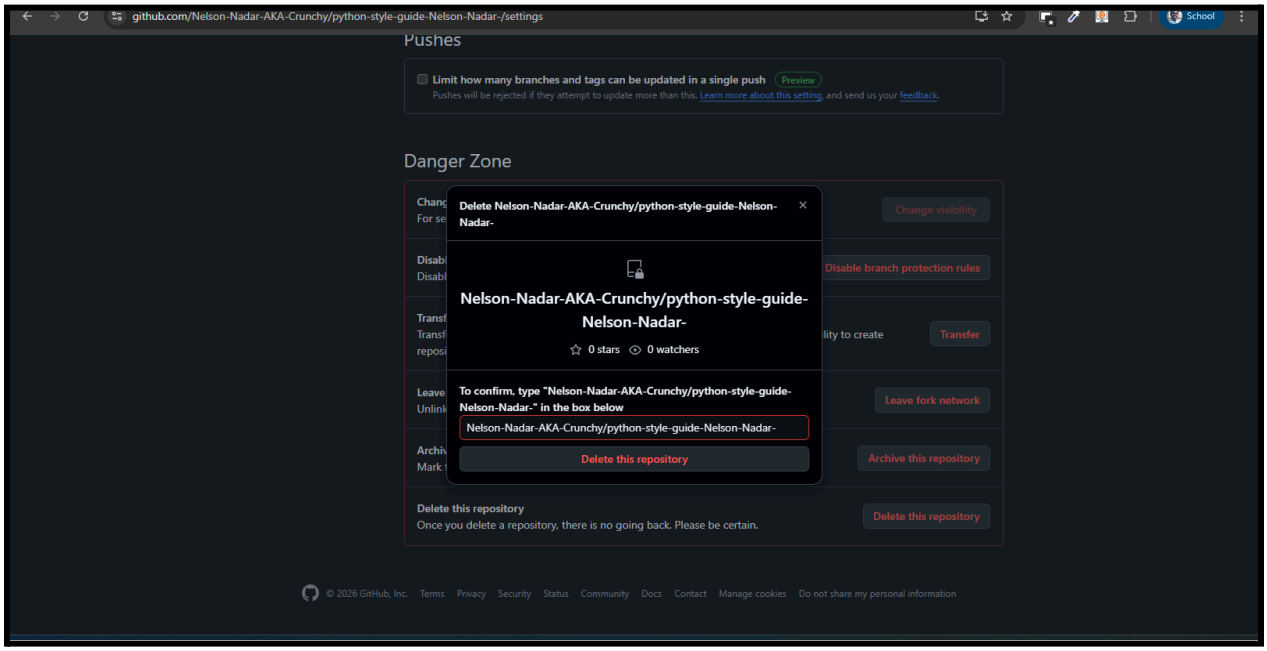


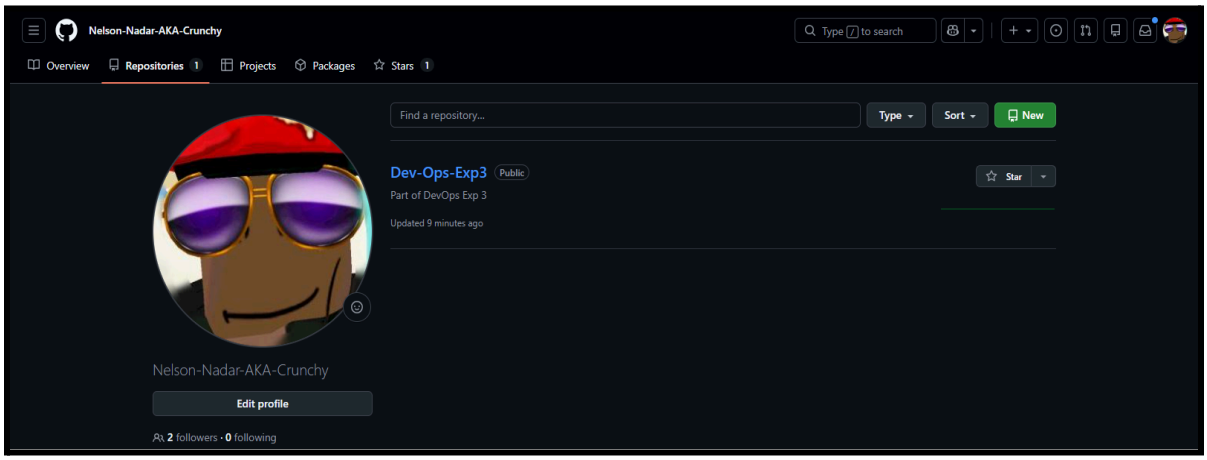
2. Fork a public repository





3. Delete repository on github





4. Git cheat sheet commands:

1. git branch

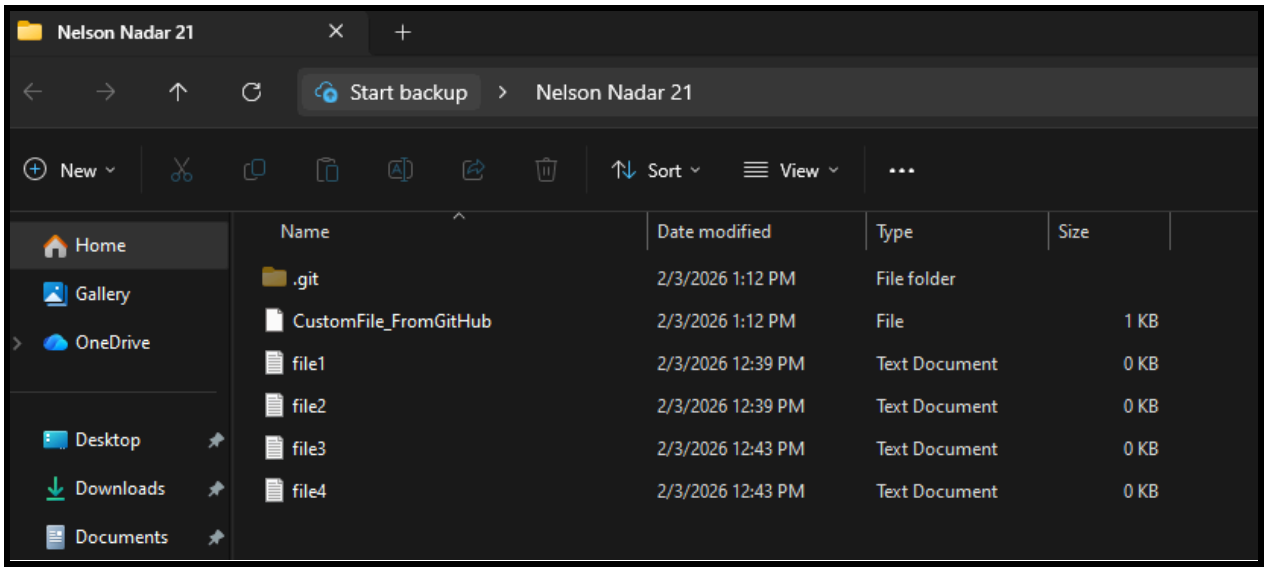
```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git branch
* master

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git branch -d Features
error: branch 'Features' not found

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git branch Features

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git branch
  Features
* master
```

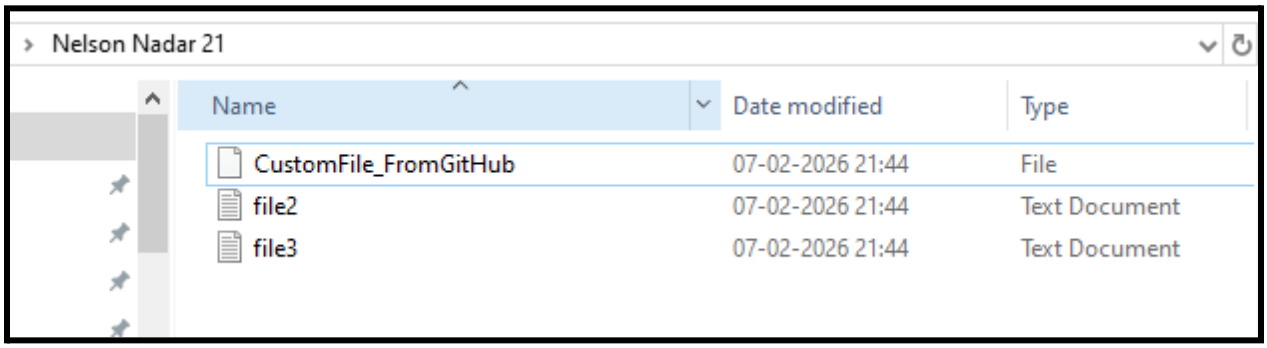
2. git checkout

```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git checkout Features
Switched to branch 'Features'

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (Features)
$ git branch
* Features
  master

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (Features)
$ git checkout master
Switched to branch 'master'

Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git branch
  Features
* master
```

3. git rm filename

```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git rm file1.txt
rm 'file1.txt'
```





4. git remote -v

```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git remote -v
origin  https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git (fetch)
origin  https://github.com/Nelson-Nadar-AKA-Crunchy/Dev-Ops-Exp3.git (push)
```

5. Any 2 other commands from git cheat sheet

- git show

```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git show c7360926c57cdc50a64cdf4bb6b767af526b09a8
commit c7360926c57cdc50a64cdf4bb6b767af526b09a8
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Feb 3 12:49:33 2026 +0530

    First Commit

diff --git a/file1.txt b/file1.txt
new file mode 100644
index 0000000..e69de29
diff --git a/file2.txt b/file2.txt
new file mode 100644
index 0000000..e69de29
```

- git log –follow [file]

```
Nelson@DESKTOP-BIEB3BM MINGW64 ~/Desktop/Nelson Nadar 21 (master)
$ git log --follow file2.txt
commit c7360926c57cdc50a64cdf4bb6b767af526b09a8
Author: Nelson Puthuraja Nadar <crunchycurry69@student.sfit.ac.in>
Date:   Tue Feb 3 12:49:33 2026 +0530

    First Commit
```